

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

SISTEMA WEB DE MONITOREO Y GESTIÓN DE LA INFRAESTRUCTURA DE REDES - ESPOCH

DOCUMENTO DE METÁFORA Y STACK TECNOLÓGICO

Arquitectura de Software, Ecosistema de Herramientas y Justificación Técnica

Integrantes del Equipo:

- William Chisag
- Licet Ibarra
- Karen Maje
- Alejandro Rodríguez

Asignaturas y Docentes Responsables:

- Infraestructura TI: Ing. Tatiana Landivar
- Tecnología Web: Ing. Pedro Aguilar
- Conmutación y Enrutamiento: Ing. Juan Pazmiño
- Big Data: Ing. Geovanny Silva
- Diseño y Multimedia: Ing. Ramiro Bonilla

Período Académico: Quinto Semestre (PAO 5)

Documento Oficial de Ingeniería de Software (Versión 1.0)

Orellana - Ecuador

Octubre de 2026

Índice general

1. Metáfora Arquitectural del Sistema **3**

1.1. Concepto de la Metáfora: La Torre de Control de Red y el Centinela 3

2. Desglose Exhaustivo del Stack Tecnológico **5**

2.1. Capa de Presentación: Frontend Web 5

2.2. Capa de Servicios y Lógica de Negocio: Backend API REST 6

2.3. Capa de Monitoreo y Exploración de Red: Daemon Centinela 6

2.4. Capa de Persistencia: Base de Datos NoSQL 7

2.5. Capa de Infraestructura, Red y Despliegue en Laboratorio 8

3. Matriz de Justificación Técnica y Alternativas Descartadas **9**

4. Articulación con las Asignaturas del Semestre (PAO 5) **11**

5. Conclusiones **12**

Índice de tablas

3.1. Matriz comparativa de selección tecnológica 10

4.1. Mapeo de tecnologías con asignaturas curriculares 11

Capítulo 1

Metáfora Arquitectural del Sistema

1.1 Concepto de la Metáfora: La Torre de Control de Red y el Centinela

En las metodologías ágiles de ingeniería de software (como eXtreme Programming y Scrum), la *metáfora del sistema* es una visión compartida de alto nivel que permite a todos los desarrolladores, evaluadores y usuarios comprender intuitivamente cómo opera el software, sus componentes y su flujo de datos sin perderse en detalles de bajo nivel.

Para el presente proyecto, la metáfora rectora seleccionada es:

"La Torre de Control de Tráfico Aéreo y el Escuadrón Centinela"

Bajo esta analogía operativa:

- **La Torre de Control (Frontend React):** Es la cabina panorámica donde los controladores (Administradores y Operadores de red) supervisan en tiempo real la posición, altura y estado de cada aeronave mediante pantallas gráficas intuitivas, radares de topología y un panel de alertas. La torre no sale a volar ni interroga directamente a los aviones en el aire; simplemente consume la telemetría depurada que recibe del centro de radar.
- **El Centro de Operaciones y Despacho (Backend Node.js / Express API):** Es la central de procesamiento que aplica las reglas de seguridad, valida autorizaciones de despegue y aterrizaje, procesa las peticiones de los controladores y expone interfaces estandarizadas (API REST) para entregar datos consolidados sin sobrecargar los sistemas de vuelo.
- **El Escuadrón Centinela de Radar (Daemon de Monitoreo en Segundo Plano):** Es un servicio autónomo que patrulla incansablemente el espacio aéreo (la red de área local). Emite pulsos periódicos de radar (pings ICMP y barridos ARP/Nmap) para descubrir nuevas naves en el perímetro y verificar que los aviones autorizados sigan respondiendo. Si una nave no responde tras tres pulsos sucesivos, el radar no asume que se desintegró físicamente, sino que emite una notificación de emergencia a la torre.
- **La Bitácora de Vuelo y Caja Negra (Base de Datos MongoDB):** Es el registro inalterable donde se almacenan las coordenadas de cada aeronave, los planes de vuelo históricos, los eventos de aproximación y las incidencias detectadas para auditoría posterior y análisis estadístico a largo plazo.

Esta metáfora resuelve de forma natural la separación de responsabilidades exigida en el requerimiento RNF-03: *el radar centinela continúa vigilando el espacio aéreo incluso cuando los operadores de la torre de control cierran sus ventanas de visualización*.

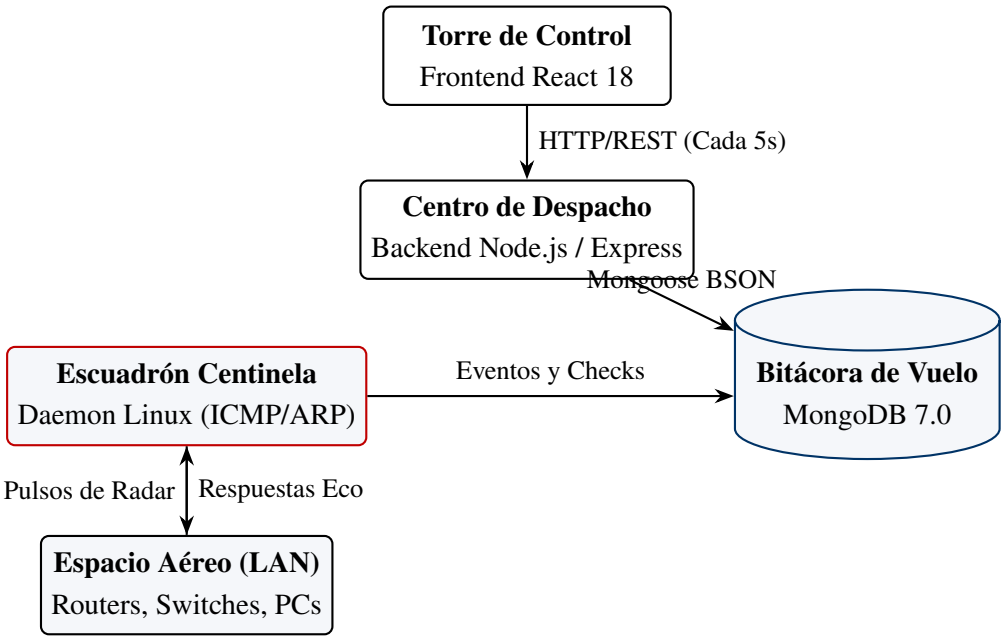


Figura 1.1: Diagrama esquemático de la metáfora arquitectural del sistema

Capítulo 2

Desglose Exhaustivo del Stack Tecnológico

El sistema implementa una arquitectura modular de tres niveles desacoplados más un servicio de fondo nativo en el sistema operativo del servidor, alineándose con las directrices de la norma ISO/IEC 25010 en cuanto a modularidad, mantenibilidad y eficiencia del desempeño.

2.1 Capa de Presentación: Frontend Web

La interfaz gráfica de usuario está orientada a la operación intuitiva en salas de control y centros de soporte de laboratorio (NOC). Debe responder fluidamente en pantallas de escritorio con resolución igual o superior a 1366×768 píxeles (RNF-10).

- **Librería Base: React 18.2+**. Permite la reactividad instantánea del estado mediante su modelo de Virtual DOM concurrente, facilitando actualizaciones selectivas del panel de control sin provocar parpadeos ni recargas completas de la página web.
- **Herramienta de Construcción y Empaquetado: Vite 5.x**. Se selecciona Vite en reemplazo del obsoleto Create React App (CRA). Vite utiliza *esbuild* basado en Go para proporcionar un arranque en desarrollo prácticamente instantáneo y empaqueta el código de producción con *Rollup*, reduciendo drásticamente el tamaño del bundle Javascript descargado por el navegador.
- **Enrutamiento del Cliente: React Router DOM v6**. Permite la navegación desacoplada mediante rutas protegidas por rol (`ProtectedRoute`), redirigiendo a los operadores lejos de las vistas exclusivas de administración sin requerir solicitudes adicionales de páginas HTML al servidor.
- **Renderizado del Grafo de Topología: React Flow (xyflow)**. Librería especializada en diagramación de grafos dinámicos e interactivos en React. Permite representar nodos con íconos personalizados de hardware (switches, routers, terminales), dibujar aristas con enlaces continuos (confirmados físicamente) o discontinuos (inferidos lógicamente), soportando controles de zoom, arrastre y selección rápida para inspección de fichas.
- **Estilizado y Sistema de Diseño: Tailwind CSS v3.4**. Framework utilitario que garantiza consistencia visual absoluta, cumplimiento riguroso de la paleta de colores del proyecto sin colisiones de nombres de clase, y fácil implementación de diseño responsivo adaptado a las pantallas de los laboratorios.
- **Iconografía Profesional: Lucide React**. Set moderno, limpio y vectorial de íconos que garantizan que el estado de cada nodo no se transmita únicamente mediante color, sino acompañado de símbolos inequívocos de comprobación, advertencia y fallo (RNF-11).
- **Cliente de Comunicación HTTP y Caché: Axios y TanStack Query (React Query)**.

Gestiona el sondeo periódico (*polling*) del panel cada 5 segundos de forma controlada, evitando fugas de memoria, manejando cancelaciones automáticas ante desmontaje de componentes e interceptando respuestas HTTP 401 para la renovación transparente de tokens.

2.2 Capa de Servicios y Lógica de Negocio: Backend API REST

El servidor de aplicaciones procesa las transacciones, gestiona la seguridad de acceso y sirve como pasarela formal entre la interfaz de usuario y la persistencia de datos.

- **Entorno de Ejecución: Node.js v20 LTS (Iron)**. Plataforma basada en el motor V8 de Google que maneja la concurrencia mediante un bucle de eventos asíncrono no bloqueante (*Event Loop*). Resulta idóneo para aplicaciones orientadas a E/S intensiva de red, donde múltiples peticiones de consulta coexisten con escrituras concurrentes de monitoreo.
- **Framework de Enrutamiento: Express.js v4.19+**. Microframework ligero y minimalista que ofrece una capa robusta de middlewares para la autenticación, control de accesos basados en roles (RBAC), registro de peticiones (*Morgan*) y manejo centralizado de excepciones.
- **Seguridad Criptográfica de Credenciales: bcrypt (12 rondas de salting)**. Algoritmo de hash adaptativo que protege las contraseñas contra ataques de fuerza bruta y tablas arcoíris. No se almacenan contraseñas en texto plano bajo ninguna circunstancia (RNF-05).
- **Gestión de Sesiones Seguras: jsonwebtoken (JWT)**. Arquitectura dual de autenticación:
  - *Access Token:* JWT de corta vigencia (15 minutos), firmado con HMAC-SHA256, transportado en la cabecera HTTP `Authorization: Bearer`.
  - *Refresh Token:* Credencial de larga duración (8 horas), persistida en base de datos y entregada al cliente exclusivamente mediante una cookie con banderas `HttpOnly`, `Secure` y `SameSite=Lax`, mitigando de raíz ataques XSS y CSRF (Sección 8.3 de la especificación).
- **Validación de Datos de Entrada: Zod / Joi**. Validación rigurosa de esquemas en servidor. Impide la inyección de tipos arbitrarios y garantiza que los parámetros de configuración (umbrales, timeouts, intervalos) se mantengan estrictamente dentro de los límites seguros (*guardrails*) definidos en la especificación RF-10.

2.3 Capa de Monitoreo y Exploración de Red: Daemon Centinela

El proceso de monitoreo opera de forma totalmente independiente del servidor web y de las sesiones activas en el navegador (RNF-03).

- **Arquitectura del Proceso:** Servicio nativo en segundo plano orquestado mediante **Systemd** en el servidor Debian del laboratorio. Si el proceso sufriera una falla inesperada, Systemd lo reinicia automáticamente en menos de 5 segundos con política `Restart=always`.

- **Mecanismos de Descubrimiento de Red (Capa 2 y Capa 3):**
  - **arp-scan:** Herramienta nativa de Linux que envía tramas ARP a nivel de Capa de Enlace (Ethernet). Permite mapear instantáneamente todos los dispositivos activos en el segmento local del laboratorio, obteniendo su dirección IP y su dirección MAC física real de hardware, incluso si el dispositivo tiene activado un firewall personal que bloquea paquetes ICMP.
  - **Nmap (Network Mapper):** Empleado para barridos de subred configurados (`nmap -sn`) y para resolución pasiva de fabricantes mediante consultas a la base de datos de identificadores OUI del IEEE.
- **Mecanismos de Comprobación de Disponibilidad:**
  - **Pings ICMP de baja sobrecarga:** Ejecución de comprobaciones de eco con tiempo de espera configurable (predeterminado 3 segundos, timeout estricto de 1 paquete).
  - **Sondeo de puertos TCP (Layer 4 fallback):** Comprobación de sockets TCP a puertos estándar (ej. 80, 443, 22) para servidores que no responden a eco ICMP pero prestan servicios de red activos.
- **Control de Concurrencia y Consumo de Recursos (RNF-15):** Limitación estricta de concurrencia mediante un pool de trabajadores asíncronos (`p-limit`) configurado a un máximo de 15 comprobaciones simultáneas. Esto impide la saturación de sockets de red y el agotamiento de descriptores de archivos en el núcleo de Linux.

2.4 Capa de Persistencia: Base de Datos NoSQL

La capa de datos utiliza un motor flexible para acomodar la naturaleza heterogénea de los equipos de red (hosts con IP, switches no administrables sin IP, puntos de acceso con múltiples clientes).

- **Motor de Base de Datos: MongoDB 7.0 Community Edition**. Sistema de base de datos documental orientado a JSON/BSON. Permite almacenar atributos dinámicos por tipo de hardware sin requerir complejas migraciones de esquema relacional.
- **Mapeador Objeto-Documento (ODM): Mongoose v8**. Facilita la declaración de esquemas estrictos, validación de tipos a nivel de aplicación, definición de hooks (*pre-save* y *post-save*) para auditoría y tipado consistente.
- **Índices TTL (Time-To-Live) para Gestión de Capacidad:** Mecanismo automático nativo de MongoDB para depurar datos de alta frecuencia y prevenir el desbordamiento de disco en el servidor Debian (Sección 8.4 de la especificación):
  - Colección `checks`: Expira automáticamente a los 30 días.
  - Colección `events`: Expira automáticamente a los 90 días (semestre académico).
  - Colección `sessions`: Expira automáticamente a las 24 horas tras caducar el token.
- **Módulo de Agregación para Big Data:** Consultas analíticas diarias mediante el *Aggregation Pipeline* de MongoDB para computar estadísticas consolidadas de disponibilidad (RF-11), resumiendo miles de pings en un único registro diario por dispositivo.

2.5 Capa de Infraestructura, Red y Despliegue en Laboratorio

- **Sistema Operativo del Servidor: Debian GNU/Linux 12 (Bookworm) x86_64**. Reconocido por su estabilidad inquebrantable, bajo consumo de memoria RAM y paquetes de red nativos maduros.
- **Servidor Web y Proxy Inverso: Nginx 1.22+**. Actúa como puerta de enlace frontal en el puerto 80/443 del servidor. Entrega los archivos estáticos optimizados del frontend React y redirige las rutas `/api/*` hacia el backend de Node.js en `localhost:5000`. Aísla completamente la base de datos MongoDB en el puerto local `127.0.0.1:27017`, impidiendo cualquier acceso directo desde el exterior.
- **Acceso Remoto Seguro (Fase 2): WireGuard VPN**. Protocolo de red privada virtual criptográfica de última generación que opera en el espacio de kernel de Linux. Permite a los docentes y evaluadores acceder al panel de control desde fuera del laboratorio a través de un túnel cifrado ChaCha20-Poly1305, sin exponer puertos del sistema a Internet ni comprometer la seguridad institucional.

Capítulo 3

Matriz de Justificación Técnica y Alternativas Descartadas

Para cumplir con los estándares de rigor técnico en la toma de decisiones de ingeniería de software, se documenta la evaluación comparativa frente a tecnologías alternativas.

Tabla 3.1: Matriz comparativa de selección tecnológica

Capa / Componente	Tecnología Elegida	Alternativas Evaluadas	Criterio y Justificación de Decisión
Frontend Web	**React 18 + Vite**	Angular, Vue.js, Django Templates	React cuenta con el ecosistema de visualización de grafos más maduro (React Flow). Vite ofrece compilaciones ultrarrápidas y menor consumo de memoria que Angular o Webpack.
Backend & API	**Node.js + Express**	Python (Django/FastAPI), Java (Spring Boot)	El modelo de I/O asíncrono no bloqueante de Node.js es superior al manejar cientos de pings y comprobaciones concurrentes de red sin levantar hilos pesados de sistema operativo. Permite compartir tipos y modelos en Javascript con el frontend.
Base de Datos	**MongoDB 7.0**	PostgreSQL, MySQL, SQLite	La estructura de los dispositivos de red es altamente polimórfica (un router tiene tablas de rutas; un switch tiene puertos y VLANs; una PC no). Los documentos BSON permiten modelar esta jerarquía sin tablas puente artificiales. Índices TTL automáticos nativos.
Daemon de Red	**Node.js Daemon + arp-scan/ping**	Python Scapy, C/C++ raw sockets	‘arp-scan‘ y ‘ping‘ son binarios del sistema altamente optimizados por el kernel de Linux. Integrarlos con Node.js proporciona equilibrio perfecto entre velocidad de ejecución en bajo nivel y facilidad de lógica de negocio.
Despliegue	**Systemd + Nginx en Debian**	Docker Compose en VM, Kubernetes	En la máquina física del laboratorio asignada para el proyecto, la virtualización o capas de abstracción pesadas de contenedores consumen memoria RAM innecesaria. La ejecución como servicio nativo Systemd garantiza el 100 % del rendimiento del hardware.

Capítulo 4

Articulación con las Asignaturas del Semestre (PAO 5)

El stack tecnológico seleccionado no es fortuito; ha sido diseñado de manera interdisciplinaria para integrar las competencias prácticas exigidas por las asignaturas de la carrera:

Tabla 4.1: Mapeo de tecnologías con asignaturas curriculares

Asignatura y Docente	Componentes del Stack Aplicados	Aporte Específico al Proyecto
Infraestructura TI (Ing. Tatiana Landivar)	Servidor Debian 12, Daemon Systemd, Nginx, Gobernanza de Servidores	Despliegue de servicios en Linux, gestión de dependencias, automatización de tareas en segundo plano y políticas de seguridad perimetral del host.
Tecnología Web (Ing. Pedro Aguilar)	React 18, Vite, Node.js, Express, REST API, JWT dual tokens	Arquitectura cliente-servidor desacoplada, consumo reactivo de endpoints, protección de sesiones web y diseño modular en capas.
Conmutación y Enrutamiento (Ing. Juan Pazmiño)	arp-scan, ICMP, sondeos TCP, Capa 2/3, Nodos Parent-Child	Comprensión de tramas Ethernet broadcast, direccionamiento IP, pasarelas por defecto, tablas de conmutación y correlación de fallos de red.
Big Data (Ing. Geovanny Silva)	MongoDB, Índices TTL, Agregación Downsampling, Métricas históricas	Procesamiento de grandes volúmenes de telemetría de red, agregación diaria de disponibilidad y retención eficiente sin desbordamiento de almacenamiento.
Diseño y Tecnología Multimedia (Ing. Ramiro Bonilla)	React Flow, Tailwind CSS, Lucide Icons, WCAG 2.1 AA	Visualización de grafos interactivos de topología, ergonomía visual, centro de notificaciones con accesibilidad cromática y acústica.

Capítulo 5

Conclusiones

La selección del stack tecnológico presentado responde a criterios de ingeniería rigurosos: rendimiento en tiempo real, mínimo consumo de recursos en hardware local, modularidad estricta y cumplimiento integral de las normas de calidad ISO/IEC 25010 y de seguridad ISO/IEC 27001. Este documento constituye la base técnica vinculante sobre la cual se desarrollará la solución durante el período académico.